Financial Trading & Deep Reinforcement Learning

Georgios Rousomanis

May 2026

Contents

1 Introduction

Deep Reinforcement Learning (DRL) has emerged as a powerful approach for solving sequential decision-making problems by combining reinforcement learning with deep neural networks. Its ability to learn complex policies directly from high-dimensional observations has led to applications across various domains, including robotics, control systems, games, and financial trading.

This work presents the development of a DRL framework for multi-asset trading in stock and cryptocurrency markets. The framework supports the complete DRL pipeline, including environment modeling, data preprocessing and normalization, policy network design, training using algorithms such as PPO and A2C, and evaluation through backtesting on unseen data.

The objective of this project is to provide a modular framework for studying and evaluating DRL methods in financial environments while enabling experimentation with different architectures, policies, and training configurations.

The implementation of this work is based on the FinRL framework.

2 Trading Environment Modeling

The proposed DRL framework is built upon custom `Gymnasium` environments that formulate the trading problem as a sequential decision-making task. At each timestep, the agent observes the current market state and portfolio information, and outputs continuous actions that determine asset allocation decisions. Two environments are implemented: a cryptocurrency trading environment and a stock trading environment.

2.1 Crypto Trading Environment

The crypto trading environment models a multi-asset portfolio management problem, where the agent interacts with market observations over time and learns trading decisions under realistic portfolio constraints. At each timestep t, the environment state is composed of two components: the portfolio state and the market state.

The portfolio state contains the available cash balance together with the current holdings for all assets:

$$s_t^{portfolio} = [C_t, H_1, H_2, \ldots, H_N]$$

where N denotes the number of traded assets, C_t is the available cash at timestep t, and H_i represents the number of held shares (or units) of asset i.

The market state contains the current prices and technical indicators for all assets:

$$s_t^{market} = [p_1, p_2, \ldots, p_N, \mathbf{tech}_1, \mathbf{tech}_2, \ldots, \mathbf{tech}_N]$$

where p_i is the current price of asset i, while

$$\mathbf{tech}_i = [t_{i,1}, t_{i,2}, \ldots, t_{i,F}] \in \mathbb{R}^F$$

is the feature vector containing the F technical indicators associated with asset i. In practice, these indicators are flattened into a single vector and concatenated with the asset prices to form the market representation. Therefore, the market state combines both raw market information (prices) and engineered features (technical indicators).

The complete environment state at timestep t is then defined as

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

thus providing the agent with information about both its internal portfolio status and the external market conditions required for decision making.

The action space is continuous and bounded in $[-1, 1]^N$, where each action component a_i represents a trading signal for asset i. Negative values indicate selling pressure, while positive values indicate buying pressure.

For sell actions ($a_i < 0$), the environment liquidates a fraction of the currently held position proportional to the action magnitude:

$$q_i^{sell} = \min(H_i,\, -a_i H_i)$$

Therefore, $a_i = -1$ corresponds to full liquidation, while intermediate values produce partial sell orders.

For buy actions ($a_i > 0$), the action determines a desired allocation relative to the current portfolio value:

$$V_t = C_t + \sum_{j=1}^{N} H_j p_j$$

$$q_i^{desired} = \frac{a_i V_t}{p_i}$$

The final purchased quantity is constrained by available cash, transaction costs and maximum exposure limits:

$$q_i^{buy} = \min\left(q_i^{desired}, \frac{C_t}{p_i(1+\kappa)}, \frac{\max(0, \lambda_i V_t - H_i p_i)}{p_i}\right)$$

where κ denotes transaction costs and λ_i defines the maximum portfolio allocation allowed for asset i. This prevents excessive concentration of capital into individual assets and enforces diversification constraints.

An alternative formulation considered during development was direct portfolio allocation, where actions lie in $[0, 1]^N$ and represent target portfolio weights. However, this approach was discarded because it requires enforcing normalization constraints at each step and couples all asset decisions through the allocation vector. The adopted action representation allows independent asset-level decisions and scales more naturally to high-dimensional multi-asset portfolios.

The reward is defined as the logarithmic portfolio growth between consecutive timesteps:

$$r_t = \log\left(\frac{V_{t+1} + \epsilon}{V_t + \epsilon}\right)$$

Logarithmic returns were preferred over simple returns because they improve numerical stability, naturally account for compounding effects, and align the optimization process with long-term portfolio growth objectives.

Overall, the environment formulates crypto trading as a continuous-control MDP where the agent must maximize portfolio growth while accounting for transaction costs, capital constraints and position limits.

2.2 Stock Trading Environment

The stock trading environment extends the crypto formulation by incorporating additional market risk information and stricter trading constraints that better reflect traditional equity markets. Similar to the crypto environment, the state consists of a portfolio state and a market state, but the market representation is augmented with volatility and turbulence signals.

The portfolio state remains identical:

$$s_t^{portfolio} = [C_t, H_1, H_2, \ldots, H_N]$$

where C_t denotes the available cash and H_i the number of held shares of stock i.

The market state extends the previous formulation by including external market risk indicators:

$$s_t^{market} = [p_1, \ldots, p_N, \mathbf{tech}_1, \ldots, \mathbf{tech}_N, v_t, \tau_t]$$

where v_t denotes the market volatility index (VIX) at timestep t and τ_t represents the turbulence level of the market. The complete environment state is therefore given by

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

allowing the agent to observe not only asset-level information but also global market risk conditions.

The action space remains continuous and bounded in $[-1, 1]^N$, where each component a_i defines the trading signal for stock i. Sell actions reduce existing positions proportionally:

$$q_i^{sell} = \min(H_i, -a_i H_i), \qquad a_i < 0$$

while buy actions determine the desired position size relative to the current portfolio value

$$V_t = C_t + \sum_{j=1}^{N} H_j p_j$$

$$q_i^{desired} = \left\lfloor \frac{a_i V_t}{p_i} \right\rfloor$$

Unlike the crypto environment, stock holdings are constrained by an explicit maximum position limit per asset. Therefore, the executed order becomes

$$q_i^{buy} = \min\left(q_i^{desired}, \left\lfloor \frac{C_t}{p_i(1+\kappa)} \right\rfloor, M_i - H_i\right)$$

where κ denotes transaction costs and M_i the maximum allowable holdings for stock i. This prevents unlimited accumulation of positions and enforces realistic exposure limits.

An additional risk-control mechanism is introduced through turbulence filtering. When market turbulence exceeds a predefined threshold τ_{thr},

$$\tau_t > \tau_{thr}$$

all positive actions are suppressed according to

$$a'_t = \min(a_t, 0)$$

forcing the agent to either maintain or reduce positions during periods of elevated market stress.

The reward extends the logarithmic portfolio growth formulation by incorporating a volatility penalty term:

$$r_t = \log\left(\frac{V_{t+1} + \epsilon}{V_t + \epsilon}\right) - \lambda e_t v_t$$

where

$$e_t = \frac{\sum_{i=1}^{N} H_i p_i}{V_t + \epsilon}$$

represents portfolio exposure, v_t is the current market volatility signal (VIX), and λ controls the strength of the risk penalty. Consequently, portfolios with high market exposure become increasingly penalized during volatile periods.

Overall, the stock environment formulates equity trading as a risk-aware continuous-control MDP in which the agent must maximize long-term portfolio growth while accounting for trading costs, position constraints, volatility exposure, and market turbulence.

3 Policy Network Architectures

Financial markets exhibit strong temporal dependencies, non-stationary behavior, and long-range interactions across assets and indicators. Consequently, the policy network must not only learn the mapping from states to actions, but also capture temporal market structure.

To investigate the effect of temporal modeling in DRL trading, this work evaluates four policy families: standard MLP policies, recurrent LSTM policies, and two custom Transformer-based feature extractors integrated into the Stable-Baselines3 framework.

Standard MLP policies treat each observation independently:

$$a_t = \pi(s_t)$$

thus ignoring historical information. While computationally efficient, this assumption is restrictive in financial environments where decisions depend on previous market states.

LSTM-based policies extend this formulation by introducing a hidden state:

$$h_t = f(h_{t-1}, s_t)$$

allowing information propagation through time. However, long temporal dependencies must be compressed into a fixed-size memory representation, which may lead to information loss.

Transformer architectures overcome this limitation through self-attention, allowing each timestep to directly interact with all previous observations inside a lookback window. This enables explicit modeling of delayed market reactions, regime changes and cross-asset dependencies, making Transformers particularly suitable for multi-asset trading tasks.

3.1 Transformer Feature Extractor (Full Fusion Architecture)

The first proposed architecture is a fully Transformer-based feature extractor where portfolio and market information are processed jointly.

For each timestep t, the observation consists of the concatenation of portfolio state and market features:

$$x_t = [s_t^{portfolio}, s_t^{market}]$$

A temporal window of length T is constructed:

$$X = [x_{t-T+1}, \ldots, x_t]$$

which forms the Transformer input.

Each observation is projected into a latent representation of dimension $d_{model} = 128$:

$$z_t = W_{proj} x_t$$

followed by addition of learnable positional embeddings:

$$\tilde{z}_t = z_t + p_t$$

to preserve temporal ordering.

The resulting sequence is processed using a Transformer encoder composed of two encoder layers with four attention heads, feedforward dimension 256, pre-layer normalization, and dropout 0.1.

The final latent representation is obtained from the output corresponding to the most recent timestep:

$$h_t = Transformer(X)_T$$

followed by Layer Normalization.

This representation h_t is used as a compact feature vector and is passed to the actor and critic networks of the underlying DRL algorithm to produce the policy distribution and value estimates.

This architecture performs full feature fusion, allowing attention to jointly model portfolio evolution, market dynamics and cross-asset interactions within a unified representation space.

3.2 Hybrid MLP–Transformer Architecture

The second architecture introduces an explicit separation between sequential market information and instantaneous portfolio state.

The market branch receives only market observations over the temporal window:

$$X^{market} = [s^{market}_{t-T+1}, \ldots, s^{market}_t]$$

which are projected to $d_{model} = 128$, combined with learnable positional embeddings and processed by the same Transformer encoder configuration used in the previous model.

The resulting market embedding is extracted from the final timestep output:

$$h^{market}_t = Transformer(X^{market})_T$$

In parallel, portfolio information is processed independently using an MLP branch. The input consists only of the most recent portfolio state:

$$h^{portfolio}_t = MLP(s^{portfolio}_t)$$

implemented using two fully connected layers of size 64 with ReLU activations.

The two latent representations are fused through concatenation:

$$z_t = [h^{market}_t, h^{portfolio}_t]$$

followed by a fully connected fusion layer projecting the representation back to $d_{model} = 128$.

The resulting fused embedding z_t is passed to the actor and critic networks of the reinforcement learning algorithm, where it is used to compute action distributions and value estimates.

This design is motivated by the different characteristics of the two information sources. Market observations are inherently sequential and benefit from temporal attention mechanisms, whereas portfolio information represents an instantaneous allocation state that does not require sequence modeling. Separating the branches reduces unnecessary coupling while preserving expressive capacity through the fusion stage.

4 Data Representation and Preprocessing for Temporal Reinforcement Learning

The Transformer-based architectures require temporally structured observations rather than isolated environment states. However, the trading environments provide observations containing information only for the current timestep:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

without historical context.

4.1 Sequence Construction via Rolling Window Buffering

To enable temporal modeling, a sequence construction mechanism is introduced through the `VecSequenceWrapper`. The wrapper maintains a rolling buffer of the most recent T observations for every parallel environment instance:

$$S_t = [s_{t-T+1}, s_{t-T+2}, \ldots, s_t]$$

where $T = 10$ in the experimental configuration.

At each environment step, the oldest observation is removed and the newest state is appended, producing a sliding temporal window. Consequently, the policy receives a sequence of past states instead of a single observation.

This mechanism is required only for temporal architectures (Transformers), since MLP and standard recurrent policies internally manage temporal information differently.

4.2 Online Rolling Window Normalization

Since financial variables exhibit substantial differences in scale (prices, holdings, portfolio values, technical indicators and rewards) and market distributions evolve continuously over time, training directly on raw observations may lead to unstable optimization and dominance of high-magnitude features.

To address this issue, a rolling normalization mechanism is introduced through the `RollingWindowNorm` wrapper. Unlike static normalization approaches computed over the entire dataset, the proposed method maintains a fixed-size temporal buffer containing the most recent observations and computes normalization statistics online.

Importantly, normalization is performed **independently for each feature dimension**. Consider an observation vector:

$$x_t = [x_t^{(1)}, x_t^{(2)}, \ldots, x_t^{(D)}]$$

where D denotes the feature dimensionality. For each feature d, the wrapper keeps a rolling history over the latest W observations:

$$\mathcal{B}_t^{(d)} = \{x_{t-W+1}^{(d)}, \ldots, x_t^{(d)}\}$$

and computes feature-specific statistics:

$$\mu_t^{(d)} = \frac{1}{W} \sum_{k=t-W+1}^{t} x_k^{(d)}$$

$$\sigma_t^{(d)} = \sqrt{\frac{1}{W} \sum_{k=t-W+1}^{t} (x_k^{(d)})^2 - (\mu_t^{(d)})^2}$$

The normalized observation is then obtained element-wise:

$$\hat{x}_t^{(d)} = \frac{x_t^{(d)} - \mu_t^{(d)}}{\sigma_t^{(d)} + \epsilon}$$

Consequently, prices, portfolio quantities and technical indicators are normalized independently according to their own recent history rather than sharing global statistics.

Reward normalization follows the same principle using an independent rolling buffer:

$$\hat{r}_t = \frac{r_t - \mu_t^r}{\sigma_t^r + \epsilon}$$

where μ_t^r and σ_t^r are computed over recent rewards.

A key practical advantage of this approach is that it is fully **online and self-contained**: the normalization statistics are never stored or fixed after training. Instead, they are continuously recomputed during both training and evaluation. This eliminates the need for a separate preprocessing step and avoids the common issue of mismatched training and test distributions, which is particularly pronounced in financial markets due to regime shifts and non-stationarity.

This adaptive normalization strategy is therefore especially suitable for trading environments, where distributional shifts between training and testing data are expected rather than exceptional. By continuously adapting to recent market conditions, the wrapper improves stability and robustness of the learned policy under changing market regimes.

5 Reinforcement Learning Algorithms

This work evaluates five standard Deep Reinforcement Learning algorithms, covering both on-policy and off-policy families. The goal is to compare how different learning paradigms behave under the same trading environment and feature representations, while using carefully tuned hyperparameter configurations that ensure stable and fair training across methods.

5.1 On-Policy Methods

On-policy methods learn directly from data generated by the current policy, meaning that the policy is continuously updated using fresh trajectories collected from interaction with the environment. This typically leads to more stable learning dynamics, since the data distribution closely matches the policy being optimized. However, this comes at the cost of lower sample efficiency, as past experiences cannot be reused.

Advantage Actor-Critic (A2C). A2C is a synchronous actor-critic algorithm that jointly learns a policy function (actor) and a value function (critic). The critic estimates the expected return, while the actor updates the policy using the advantage signal, reducing variance compared to standard policy gradient methods.

A2C uses a rollout length of `n_steps = 128`, which controls how many environment steps are collected before each update. Smaller values lead to more frequent but noisier updates, while larger values improve gradient stability at the cost of responsiveness.

The discount factor `gamma = 0.99` determines how much the agent prioritizes long-term rewards, which is essential in financial settings where gains are often delayed. The parameter `gae_lambda = 0.95` controls the bias-variance trade-off in advantage estimation, with lower values favoring bias reduction and higher values improving variance reduction.

Exploration is encouraged via `ent_coef = 0.01`, which prevents premature convergence to deterministic policies, while `vf_coef = 0.5` balances policy learning with value function accuracy. Gradient stability is further ensured using `max_grad_norm = 0.5` to prevent excessively large updates.

Proximal Policy Optimization (PPO). PPO improves upon standard policy gradient methods by constraining policy updates through a clipped objective, preventing large and unstable policy shifts.

A large rollout size of `n_steps = 2048` is used to ensure low-variance advantage estimation before each optimization phase. The optimization itself is performed using mini-batches of size `batch_size = 64`, which provides a balance between gradient stability and computational efficiency.

The parameter `target_kl = 0.02` plays a key role in stabilizing training by limiting the Kullback-Leibler divergence between successive policies. Lower values enforce more conservative updates, while higher values allow faster but potentially unstable learning.

As in A2C, the entropy coefficient `ent_coef = 0.01` is used to maintain sufficient exploration during training, which is particularly important in non-stationary financial environments.

5.2 Off-Policy Methods

Off-policy methods learn from experiences stored in a replay buffer, allowing the agent to reuse past transitions generated by previous policies. This significantly improves sample efficiency and enables more stable learning in environments where data collection is expensive or limited. However, these methods are generally more sensitive to hyperparameter choices and function approximation errors.

Deep Deterministic Policy Gradient (DDPG). DDPG is an off-policy actor-critic method for continuous action spaces that learns a deterministic policy.

A replay buffer of size `buffer_size = 20,000` stores past transitions, enabling learning from previously experienced market conditions. The `batch_size = 256` controls the number of samples used per update, where larger values improve gradient stability but increase computational cost.

Training begins after an initial exploration phase of `learning_starts = 1000` steps, ensuring that the replay buffer contains sufficiently diverse experiences before learning commences. The learning rate `1e-5` is intentionally small to ensure stable updates in the presence of highly noisy financial signals.

Twin Delayed DDPG (TD3). TD3 extends DDPG by introducing twin critic networks, delayed policy updates, and target policy smoothing, significantly improving stability.

It shares the same core hyperparameters as DDPG, including `buffer_size = 20,000`, `batch_size = 256`, and `learning_starts = 1000`. These settings ensure a consistent comparison with DDPG while benefiting from TD3's architectural improvements that reduce overestimation bias and improve value function reliability.

Soft Actor-Critic (SAC). SAC is an off-policy method that optimizes a stochastic policy under an entropy-regularized objective, explicitly encouraging exploration.

It uses the same replay buffer configuration (`buffer_size = 20,000`) and batch size (`256`) as TD3 and DDPG for consistency. The entropy coefficient is set to `ent_coef = auto`, allowing the algorithm to automatically adjust the exploration– exploitation trade-off during training.

As with other off-policy methods, `learning_starts = 1000` ensures that early updates are performed only after sufficient exploratory data has been collected.

Overall, on-policy and off-policy methods represent two complementary design choices in DRL. On-policy algorithms rely on fresh, on-distribution data and typically offer more stable and predictable learning dynamics, but at the cost of higher sample requirements. Off-policy methods instead prioritize sample efficiency by reusing past experiences through replay buffers, which is particularly beneficial in financial environments where data is expensive and sequential

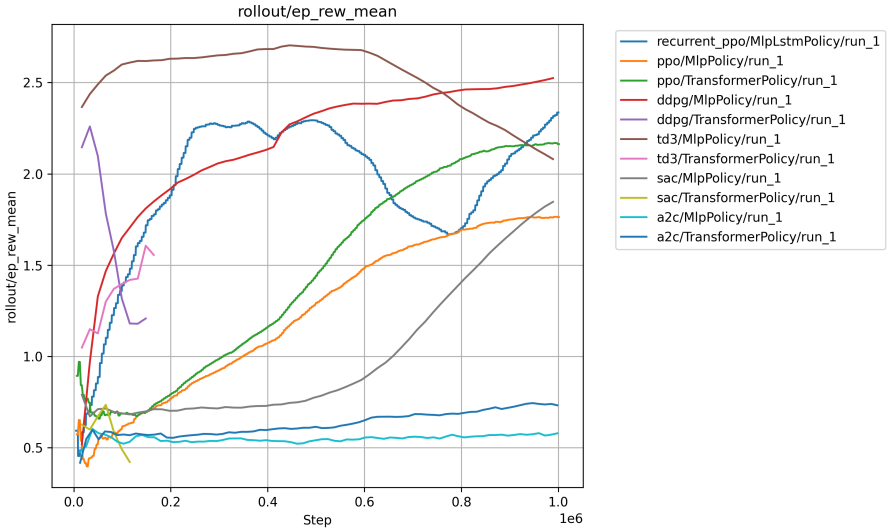

Figure 1: Training curves of episodic mean reward (`ep_rew_mean`) across all evaluated DRL algorithms and policy architectures.

correlations are strong. However, this comes with increased sensitivity to hyperparameters and potential instability due to distributional shift in stored experiences. Evaluating both families under carefully tuned configurations provides a more complete understanding of their behavior in realistic trading scenarios.

6 Training Performance and Stability Analysis

Training is performed on historical BTC-USD data using daily candles (1D frequency) covering the period from `2014-01-06` to `2025-12-31`. All models are trained for up to 1,000,000 timesteps under identical environment dynamics, reward formulation, and preprocessing pipeline, enabling direct comparison across algorithms and policy architectures.

Training behavior is analyzed using episodic mean reward, together with two financial metrics: *total return*, measuring cumulative portfolio growth, and *Sharpe ratio*, measuring risk-adjusted return. Table 1 summarizes the final reward values, while Figures 1, 2, and 3 illustrate the training evolution of all monitored metrics.

A2C. Both A2C variants remain nearly flat throughout training in Figure 1, converging to relatively low reward levels. This behavior is consistently reflected in total return and Sharpe ratio (Figures 2 and 3), indicating limited policy improvement during training. The Transformer variant provides only a marginal performance gain over the MLP baseline.

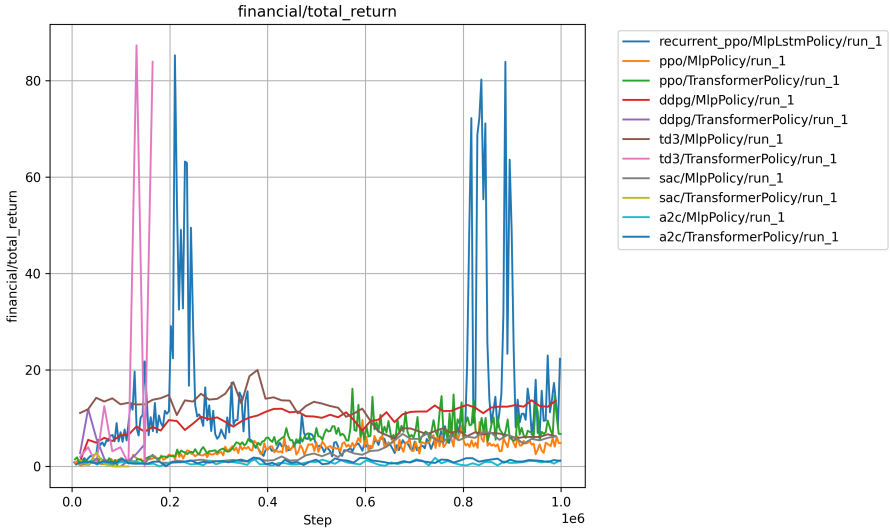

Figure 2: Episode total return across all evaluated DRL algorithms and policy architectures.

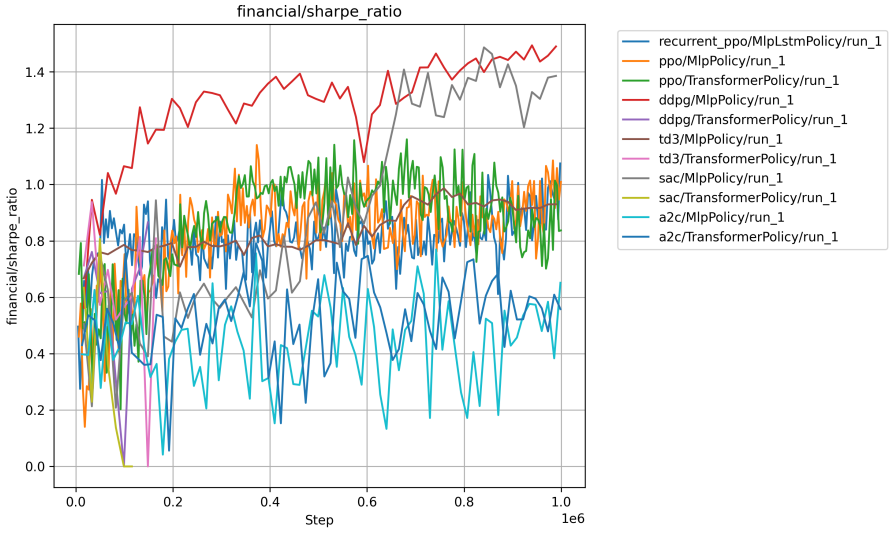

Figure 3: Sharpe ratio across all evaluated DRL algorithms and policy architectures.

Model	Smoothed Value	Final Value	Timesteps
A2C (MLP)	0.5743	0.5788	998,400
A2C (Transformer)	0.7356	0.7319	998,400
DDPG (MLP)	2.5110	2.5246	989,280
DDPG (Transformer)	1.2787	1.2076	148,392
PPO (MLP)	1.7632	1.7628	1,001,472
PPO (MLP–Transformer)	1.7386	1.7396	1,001,472
PPO (Transformer)	2.1647	2.1627	1,001,472
Recurrent PPO (LSTM)	2.3349	2.3360	1,000,448
SAC (MLP)	1.8009	1.8467	989,280
SAC (Transformer)	0.5139	0.4212	115,416
TD3 (MLP)	2.1192	2.0805	989,280
TD3 (Transformer)	1.5086	1.5557	164,880

Table 1: Final training performance across different DRL algorithms and architectures.

PPO. All PPO variants exhibit stable learning dynamics with clear convergence behavior across all monitored metrics. The Transformer policy achieves the strongest performance within the PPO family (Table 1), while also maintaining strong total return and Sharpe ratio behavior. The hybrid MLP–Transformer architecture performs similarly to the MLP baseline, suggesting limited benefit from partial separation of portfolio and market representations in this experimental setting.

Recurrent PPO (LSTM). Recurrent PPO achieves the highest episodic reward (Table 1); however, the reward trajectory is non-monotonic and exhibits noticeable oscillations (Figure 1), indicating reduced training stability compared to standard PPO variants. This behavior is further reflected in the total return metric, where extreme positive spikes approaching numerical divergence appear (Figure 2), suggesting episodic instability under long-horizon portfolio compounding.

DDPG. DDPG (MLP) achieves the strongest overall performance across all monitored metrics. It consistently attains the highest total return while maintaining strong Sharpe ratio behavior and stable reward progression. In contrast, the Transformer variant terminates early at approximately 148k timesteps and performs substantially worse across all metrics.

SAC. The SAC MLP model demonstrates moderate but relatively stable performance across reward, total return, and Sharpe ratio. The Transformer variant fails to complete training, terminating at approximately 115k timesteps with weaker performance and unstable learning behavior.

TD3. The TD3 MLP model exhibits non-monotonic reward dynamics, starting from relatively high reward values before gradually declining during training. The Transformer variant terminates early at approximately 165k timesteps and additionally exhibits extreme spikes in total return (Figure 2), similar to recurrent PPO, indicating episodic instability rather than sustained portfolio growth.

Overall, the results show consistent trends across reward and financial metrics. MLP policies provide the most stable training behavior, while Transformer architectures improve representation capacity for on-policy methods—particularly PPO—but reduce stability in off-policy algorithms. Recurrent policies provide strong temporal modeling capabilities, although they introduce additional instability in long-horizon portfolio dynamics.